

Multi-Source Candidate Data Transformer

10-Minute Demo Walkthrough & Speaking Script

Eightfold Engineering Intern (Jul-Dec 2026) - Assignment. A timed, read-aloud script for the demo video / live walkthrough, plus exact commands to run and a Q&A defense cheat-sheet. Every command, number, and example is drawn from the actual repository.

NORTH STAR (say this to yourself before you start)

Eightfold's real fear is stated in their own brief: "Wrong-but-confident is worse than honestly-empty." Every design choice serves that one sentence - deterministic core, provenance on every value, confidence that drops on conflict, and unknowns that stay null instead of getting invented. The whole talk is: here is how I built a system that is never confidently wrong.

At a glance

Time	Segment	What you do
0:00-1:15	Frame the problem	State the 'wrong-but-confident' rule; show the 7 sources
1:15-2:30	The pipeline	Walk detect - extract - normalize - merge - project - validate
2:30-4:30	LIVE: default run	One command - one clean profile; narrate merges + provenance
4:30-6:00	LIVE: the twist	Config-driven reshape, no code change; the lint bonus
6:00-7:15	Design decision	The LLM fence - where you deliberately did NOT use an LLM
7:15-8:45	Edge cases	CA=California, dateutil today-bleed, graceful degradation
8:45-10:00	Constraints + close	Determinism proof, scale (NDJSON), honest scope, close

Need the ~2-minute video only? Keep 2:30-4:30 (default), 4:30-5:30 (one custom config), 6:00-6:45 (the design decision), and 7:15-7:45 (the CA edge case).

0:00-1:15

Frame the problem in their words

[SAY]

The task is a multi-source candidate transformer. Eightfold ingests candidate data from many places - recruiter exports, ATS systems, GitHub, resumes, free-text notes - and downstream hiring products need one clean canonical profile per candidate: fixed fields, normalized formats, deduplicated across sources, with a record of where each value came from and how confident we are. The line that drove my entire design is this: wrong-but-confident is worse than honestly-empty, because a bad value silently pollutes hiring decisions. So I optimized for trustworthiness over completeness. The two things an LLM does - vary its output run to run, and hallucinate - are the two things this system must never do. So my core is 100% deterministic and LLM-free, and I'll show you that guarantee is enforced, not just claimed.

[SHOW]

The `samples/sources/` folder - 7 files for one candidate:

- **Structured** (brief requires ≥ 1): `john_smith_recruiter.csv`, `john_smith_ats.json`
- **Unstructured** (brief requires ≥ 1): `john_smith_github.json`, `john_smith_resume.docx`, `john_smith_notes.txt`
- **Two deliberate landmines**: `garbage_source.json`, `empty_source.csv`

Point to make: "I cover five source types - two structured, three unstructured - well past the 'one from each group' requirement. And I seeded two broken sources to prove graceful degradation."

1:15-2:30

The pipeline (how you think)

[SHOW]

The architecture diagram in the README (the detect - extract - normalize - merge - project - validate block).

[SAY]

The pipeline is six stages: detect routes each file to the right adapter; extract - one adapter per source - turns each into a flat list of field fragments; normalize happens right at extraction, so every fragment already carries its canonical value and records which normalization produced it; merge links records that are the same person and resolves conflicts; project reshapes to whatever the caller's config asks for; and validate checks the result against the requested schema. The critical architectural line is between the canonical record and the projection layer. The engine always builds the same rich internal profile; the config only controls the view on the way out. That separation is what lets one engine emit any shape with zero code changes - the 'required twist' in the brief.

2:30-4:30

LIVE: default run (the working core)

[SHOW]

Run it, then open `out.json` (or show `samples/expected/default.json`):

```
python -m transformer.cli run -i samples/sources -c default -o out.json
```

[SAY]

One command, seven messy files, one clean profile. Let me show the interesting merges, because this is where 'never confidently wrong' gets real. NAME: the ATS says John Smith, the recruiter CSV says John A. Smith. My trust order is ATS > recruiter CSV > GitHub > resume > notes, so the highest-trust source wins - output is John Smith. That conflict is recorded in provenance, and the field's confidence is lowered because sources disagreed. TITLE: same story - CSV and resume say Senior Software Engineer, ATS says Staff Software Engineer. ATS wins, conflict logged, confidence down. PHONES: these aren't a conflict - they're three different real numbers across three sources: the CSV's (415) 555-0132, the ATS's +1-415-555-0144, and a personal cell 415-555-0177 buried in the free-text notes. All three normalize to E.164 and are unioned. PROVENANCE: every retained value has an entry - (field, source, method). Forty of them here. Nothing is untraceable, including the derived candidate_id - I have a test that asserts that invariant. CONFIDENCE: overall_confidence is 0.694 - a simple, documented formula: strongest single piece of evidence, a small boost per agreeing source, and a x0.7 penalty on any conflict. It's a data-corroboration signal, explicitly not a hiring score.

4:30-6:00

LIVE: the twist (config-driven output, no code change)

[SHOW]

Open `samples/configs/recruiter_summary.json` - the brief's verbatim example.

[SAY]

Now the required twist: a runtime config that reshapes the output - same engine, no code changes. This is the exact example from the assignment. It renames fields with a path mini-language: `full_name` stays, `primary_email` comes from `emails[0]`, `phone` from `phones[0]` normalized to E.164, and `skills` becomes a flat list from `skills[].name`, canonicalized.

```
python -m transformer.cli run -i samples/sources \
    -c samples/configs/recruiter_summary.json
```

[SAY]

Notice `primary_email` is `j.smith@workmail.com`. Emails are sorted for determinism, so `[0]` is alphabetically first - not semantically 'primary'. I call that out in my README rather than hide it: the determinism guarantee being honest about a trade-off. And projection and validation are two separate steps. `on_missing` controls representation - null, omit, or error. Then validation runs after: a required field that ended up null raises a clear, field-named error, and a type mismatch raises too. The config itself is pydantic-validated with `extra='forbid'`, so a typo'd key is rejected at load.

[SHOW]

Bonus that shows judgment - the static config linter:

```
python -m transformer.cli lint samples/configs/recruiter_summary.json
```

Point to make: "I added a `lint` command that checks a config's paths against the schema before a run - so a mistyped `location.citi` is caught as an error instead of silently resolving to null."

6:00-7:15

The design decision I'm most proud of (the LLM fence)

(The demo video explicitly asks for one design decision you're proud of - this is it, and it's pure judgment.)

[SAY]

The decision I'm proudest of is where I did not use an LLM. The brief forbids exactly two things - non-determinism and invented data - and those are the two things LLMs do. So my deterministic core, the path that gets graded, makes zero LLM calls and zero network calls. But there are genuinely fuzzy prose fields - headline, and experience summary - where deterministic parsing is weak. So I built an LLM lane that is off by default and completely fenced: it runs after merge, only fills fields the core left null, never overwrites a deterministic value, and never participates in conflict resolution. It's extract-or-null - use only the provided text, return null for anything not present - schema-validated, low-confidence, provenance-tagged as llm:gemini-2.5-flash, cached for reproducible replay, and failure-isolated so a bad key or network error just leaves the field null and the run continues. I chose a Flash-tier model deliberately: the task is tiny prose extraction, so a bigger model adds cost and latency with no accuracy gain. The key is read from the environment only - never a flag, never committed. That's the judgment call: an LLM is a scalpel for two prose gaps, never a decision-maker.

[SHOW]

Optional, if a cached key is set up - replays from the committed cache, no network:

```
python -m transformer.cli run -i samples/sources --default --enrich
```

7:15-8:45

The edge case I'm proud of (+ graceful degradation)

(The demo video also asks for one edge case you handled - lead with the best.)

[SAY]

My favorite edge case: 'CA' is California, not Canada. The notes say 'based in San Francisco, CA.' - with a period. That produced a token 'CA.' which, being three characters, slipped past a length-2-only state-abbreviation guard, and the country library happily resolved it to Canada. Silently relocating a candidate's country is exactly the confidently-wrong error an ATS must never make. Fixed by stripping trailing punctuation per segment - with a test pinning it. A subtler one: dateutil bleeds today's date into partial dates. By default it fills missing components from datetime.now(), so a resume that just says '2019' would parse to a different month depending on the day you run it - a determinism leak hiding in a library default. I pass a fixed default datetime, so '2019' is always 2019-01. My property-based tests actually found a variant of that bug. And graceful degradation: the corrupt JSON gets routed to the ATS adapter, fails to parse there, and yields an empty list; the empty CSV yields empty. Every source is read in its own try/except, so one bad file never sinks the run - the other sources still produce a complete profile. A missing or garbage source must not crash - a hard constraint in the brief, tested with garbage, empty, corrupt, unreadable, and an adapter that deliberately raises.

8:45-10:00

Constraints, scope honesty, and close

[SHOW]

Prove determinism and scale live:

```
pytest -q                                # 248 tests, ruff + mypy green
python -m transformer.cli run -i samples/scale \
    -c default --format ndjson           # one candidate per line
```

[SAY]

The brief's three constraints, each addressed head-on. Deterministic and explainable: same inputs, byte-identical output. My CI proves it three ways - run twice, shuffle the input order, and run in separate processes under different PYTHONHASHSEED values. Byte-identical every time, and explainable via provenance on every field. Robust: a missing or garbage source never crashes the run; unknown values become null, never invented. Scale: the design streams - I emit NDJSON, one candidate per line, and record linking is near-linear union-find, not pairwise. On scope, honestly: the live GitHub fetch is implemented but the tested path reads a committed fixture for offline determinism; the resume sample is DOCX not PDF - both readers exist, but DOCX I can generate deterministically; and name-only record linking is a documented heuristic, guarded so it provably refuses to fuse two different 'John Smith's that carry conflicting candidate IDs - because for an ATS, under-merging is safe but wrongly merging two people is not. That's the system: 248 tests, deterministic by construction, every value traceable, honest about what it doesn't know - built around the one rule that wrong-but-confident is the worst outcome. Thank you.

The brief says *"you own every line and must be able to explain and defend it."* Likely questions and a one-line answer for each:

If they ask...	Say
Why trust ATS over the resume?	ATS data is structured and curated by the recruiting system; resume prose is self-reported and parse-lossy. The trust order is explicit and configurable, not hardcoded magic.
How do you decide two records are the same person?	Strong union-find on shared candidate_id / email / phone first; then a guarded name-attach that merges name-only sources only when there's no conflicting candidate_id. Bias is toward under-merging.
What if two people really share an email (data-entry error)?	Linking unions on it, but I emit a loud warning when the fused cluster carries two distinct candidate IDs - the worst-case error is made visible, not hidden.
Is overall_confidence a hiring score?	No - it's a data-corroboration signal (how well sources agree). I state that explicitly so no one misuses it.
Why not use the LLM to merge?	Non-determinism and hallucination - the two things the brief forbids. The LLM is fenced to gap-filling two null prose fields, extract-or-null.
Does 'Java' ever become 'JavaScript'?	Can't happen - skill matching uses a full-string scorer (token_sort_ratio), not substring, and both are exact taxonomy aliases. Typos still resolve: 'kubernets' -> Kubernetes at 94.7.
How does the confidence math work?	support = max(trust/100 x raw_conf); +0.05 per extra agreeing source, capped +0.15; x0.7 on a resolved conflict; overall = identity-weighted mean of field confidences.

Before you record

- Do a **dry run end-to-end once** first so the commands are in shell history and nothing stalls on camera.
- Have `samples/expected/default.json` open in a second pane as a fallback if a live run is slow.
- Keep the terminal font large; the JSON output should be readable on the recording.
- The two graded must-haves for the video: **default output + one custom config**, and **one design decision + one edge case** spoken in your own words.